

第 1 章 PCI 总线的基本知识

PCI 总线作为处理器系统的局部总线，其主要目的是为了连接外部设备，而不是作为处理器的系统总线连接 Cache 和主存储器。但是 PCI 总线、系统总线和处理器体系结构之间依然存在着紧密的联系。

PCI 总线作为系统总线的延伸，其设计考虑了许多与处理器相关的内容，如处理器的 Cache 共享一致性和数据完整性（Data Consistency，也称为 Memory Consistency），以及如何与处理器进行数据交换等一系列内容。其中 Cache 共享一致性和数据完整性是现代处理器局部总线的设计的重点和难点，也是本书将重点讲述的主题之一。

孤立地研究 PCI 总线并不可取，因为 PCI 总线仅是处理器系统的一个部分。深入理解 PCI 总线需要了解一些与处理器体系结构相关的知识。这些知识是本书侧重描述的，同时也是 PCI 总线规范忽略的内容。脱离实际的处理器系统，不容易也不可能深入理解 PCI 总线规范。

对于今天的读者来说，PCI 总线提出的许多概念略显过时，也有许多不足之处。但是在当年，PCI 总线与之前存在的其他并行局部总线如 ISA、EISA 和 MCA 总线相比，具有许多突出的优点，是一个全新的设计。

(1) PCI 总线空间与处理器空间隔离

PCI 设备具有独立的地址空间，即 PCI 总线地址空间，该空间与存储器地址空间通过 HOST 主桥隔离。处理器需要通过 HOST 主桥才能访问 PCI 设备，而 PCI 设备需要通过 HOST 主桥才能访问主存储器。在 HOST 主桥中含有许多缓冲，这些缓冲使得处理器总线与 PCI 总线工作在各自的时钟频率中，互不干扰。HOST 主桥的存在也使得 PCI 设备和处理器可以方便地共享主存储器资源。

处理器访问 PCI 设备时，必须通过 HOST 主桥进行地址转换；而 PCI 设备访问主存储器时，也需要通过 HOST 主桥进行地址转换。HOST 主桥的一个重要作用就是将处理器访问的存储器地址转换为 PCI 总线地址。PCI 设备使用的地址空间是属于 PCI 总线域的，这与存储器地址空间不同。

x86 处理器对 PCI 总线域与存储器域的划分并不明晰，这也使得许多程序员并没有准确地区分 PCI 总线域地址空间与存储器域地址空间。而本书将反复强调存储器地址和 PCI 总线地址的区别，因为这是理解 PCI 体系结构的重要内容。

PCI 规范并没有对 HOST 主桥的设计进行约束。每一个处理器厂商使用的 HOST 主桥，其设计都不尽相同。HOST 主桥是联系 PCI 总线与处理器的核心部件，掌握 HOST 主桥的实现机制是深入理解 PCI 体系结构的前提。

本书将以 Freescale 的 PowerPC 处理器和 Intel 的 x86 处理器为例，说明各自 HOST 主桥的实现方式，值得注意的是本书涉及的 PowerPC 处理器仅针对 Freescale 的 PowerPC 处理器，而不包含 IBM 的 Power 和 AMCC 的 PowerPC 处理器。而且如果没有特别说明，本书中涉及的 x86 处理器特指 Intel 的处理器，而不是其他厂商的 x86 处理器。

(2) 可扩展性

PCI 总线具有很强的扩展性。在 PCI 总线中, HOST 主桥可以直接推出一条 PCI 总线, 这条总线也是该 HOST 主桥管理的第一条 PCI 总线, 该总线还可以通过 PCI 桥扩展出一系列 PCI 总线, 并以 HOST 主桥为根节点, 形成 1 棵 PCI 总线树。这些 PCI 总线都可以连接 PCI 设备, 但是在 1 棵 PCI 总线树上, 最多只能挂接 256 个 PCI 设备 (包括 PCI 桥)。

在同一条 PCI 总线上的设备间可以直接通信, 而并不会影响其他 PCI 总线上设备间的数据通信。隶属于同一棵 PCI 总线树上的 PCI 设备, 也可以直接通信, 但是需要通过 PCI 桥进行数据转发。

PCI 桥是 PCI 总线的一个重要组成部件, 该部件的存在使得 PCI 总线极具扩展性。PCI 桥也是有别于其他局部总线的一个重要部件。在“以 HOST 主桥为根节点”的 PCI 总线树中, 每一个 PCI 桥下也可以连接一个 PCI 总线子树, PCI 桥下的 PCI 总线仍然可以使用 PCI 桥继续进行总线扩展。

PCI 桥可以管理这个 PCI 总线子树, PCI 桥的配置空间含有一系列管理 PCI 总线子树的配置寄存器。在 PCI 桥的两端, 分别连接了两条总线, 分别是上游总线 (Primary Bus) 和下游总线 (Secondary Bus)。其中与处理器距离较近的总线被称为上游总线, 另一条被称为下游总线。这两条总线间的通信需要通过 PCI 桥进行。PCI 桥中的许多概念被 PCIe 总线采纳, 理解 PCI 桥也是理解 PCIe 体系结构的基础。

(3) 动态配置机制

PCI 设备使用的地址可以根据需要由系统软件动态分配。PCI 总线使用这种方式合理地解决了设备间的地址冲突, 从而实现了“即插即用”功能。因此 PCI 总线不需要使用 ISA 或者 EISA 接口卡为解决地址冲突而使用的硬件跳线。

每一个 PCI 设备都有独立的配置空间, 在配置空间中含有该设备在 PCI 总线中使用的基地址, 系统软件可以动态配置这个基地址, 从而保证每一个 PCI 设备使用的物理地址并不相同。PCI 桥的配置空间中含有其下 PCI 子树所能使用的地址范围。

(4) 总线带宽

PCI 总线与之前的局部总线相比, 极大提高了数据传送带宽, 32 位/33 MHz 的 PCI 总线可以提供 132 MB/s 的峰值带宽, 而 64 位/66 MHz 的 PCI 总线可以提供的峰值带宽为 532 MB/s。虽然 PCI 总线所能提供的峰值带宽远不能和 PCIe 总线相比, 但是与之前的局部总线 ISA、EISA 和 MCA 总线相比, 仍然具有极大的优势。

ISA 总线的最高主频为 8 MHz, 位宽为 16, 其峰值带宽为 16 MB/s; EISA 总线的最高主频为 8.33 MHz, 位宽为 32, 其峰值带宽为 33 MB/s; 而 MCA 总线的最高主频为 10 MHz, 最高位宽为 32, 其峰值带宽为 40 MB/s。PCI 总线提供的峰值带宽远高于这些总线。

(5) 共享总线机制

PCI 设备通过仲裁获得 PCI 总线的使用权后, 才能进行数据传送, 在 PCI 总线上进行数据传送, 并不需要处理器进行干预。

PCI 总线仲裁器不在 PCI 总线规范定义的范围内, 也不一定是 HOST 主桥和 PCI 桥的一部分。虽然绝大多数 HOST 主桥和 PCI 桥都包含 PCI 总线仲裁器, 但是在某些处理器系统的设计中也可以使用独立的 PCI 总线仲裁器。如在 PowerPC 处理器的 HOST 主桥中含有 PCI 总线仲裁器, 但是用户可以关闭这个总线仲裁器, 而使用独立的 PCI 总线仲裁器。

PCI 设备使用共享总线方式进行数据传递，在同一条总线上，所有 PCI 设备共享同一总线带宽，这将极大地影响 PCI 总线的利用率。这种机制显然不如 PCIe 总线采用的交换结构，但是在 PCI 总线盛行的年代，半导体的工艺、设计能力和制作成本决定了采用共享总线方式是当时的最优选择。

(6) 中断机制

PCI 总线上的设备可以通过四根中断请求信号 INTA~D#向处理器提交中断请求。与 ISA 总线上的设备不同，PCI 总线上的设备可以共享这些中断请求信号，不同的 PCI 设备可以将这些中断请求信号“线与”后，与中断控制器的中断请求引脚连接。PCI 设备的配置空间记录了该设备使用这四根中断请求信号的信息。

PCI 总线还进一步提出了 MSI (Message Signal Interrupt) 机制，该机制使用存储器写总线事务传递中断请求，并可以使用 x86 处理器 FSB (Front Side Bus) 总线提供的 Interrupt Message 总线事务，从而提高了 PCI 设备的中断请求效率。

虽然从现代总线技术的角度来看，PCI 总线仍有许多不足之处，但也不能否认 PCI 总线已经获得了巨大的成功。不仅 x86 处理器将 PCI 总线作为标准的局部总线连接各类外部设备，PowerPC、MIPS 和 ARM^①处理器也将 PCI 总线作为标准局部总线。除此之外，基于 PCI 总线的外部设备，如以太网控制器、声卡、硬盘控制器等，也已经成为主流。

1.1 PCI 总线的组成结构

如上文所述，PCI 总线作为处理器系统的局部总线，是处理器系统的一个组成部件，讲述 PCI 总线的组成结构不能离开处理器系统这个大环境。在一个处理器系统中，与 PCI 总线相关的模块如图 1-1 所示。

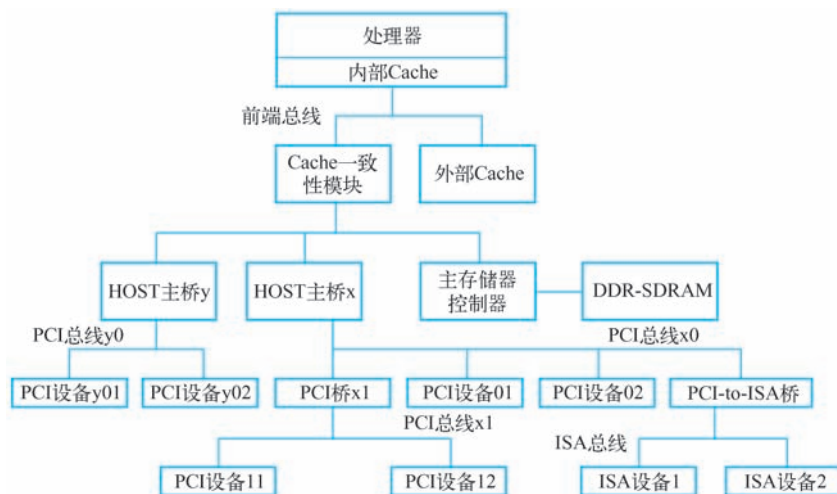


图 1-1 基于 PCI 总线的处理器系统

① 在 ARM 处理器中，使用 SoC 平台总线，即 AMBA 总线，连接片内设备。但是某些 ARM 生产厂商，依然使用 AMBA-to-PCI 桥推出 PCI 总线，以连接 PCI 设备。

图中与 PCI 总线相关的模块包括：HOST 主桥、PCI 总线、PCI 桥和 PCI 设备。PCI 总线由 HOST 主桥和 PCI 桥推出，HOST 主桥与主存储器控制器在同一级总线上，因此 PCI 设备可以方便地通过 HOST 主桥访问主存储器，即进行 DMA 操作。

值得注意的是，PCI 设备的 DMA 操作需要与处理器系统的 Cache 进行一致性操作，当 PCI 设备通过 HOST 主桥访问主存储器时，Cache 一致性模块将进行地址监听，并根据监听的结果改变 Cache 的状态。

在一些简单的处理器系统中，可能不含有 PCI 桥，此时所有 PCI 设备都是连接在 HOST 主桥推出的 PCI 总线上。此外在一些处理器系统中可能含有多个 HOST 主桥，如图 1-1 所示的处理器系统中含有 HOST 主桥 x 和 HOST 主桥 y。

1.1.1 HOST 主桥

HOST 主桥是一个很特别的桥片，其主要功能是隔离处理器系统的存储器域与处理器系统的 PCI 总线域，管理 PCI 总线域，并完成处理器与 PCI 设备间的数据交换。处理器与 PCI 设备间的数据交换主要由“处理器访问 PCI 设备的地址空间”和“PCI 设备使用 DMA 机制访问主存储器”这两部分组成。

为简便起见，下面将处理器系统的存储器域简称为存储器域，而将处理器系统的 PCI 总线域称为 PCI 总线域，存储器域和 PCI 总线域的详细介绍见第 2.1 节。值得注意的是，在一个处理器系统中，有几个 HOST 主桥，就有几个 PCI 总线域。

HOST 主桥在处理器系统中的位置并不相同，如 PowerPC 处理器将 HOST 主桥与处理器集成在一个芯片中。而有些处理器不进行这种集成，如 x86 处理器使用南北桥结构，处理器内核在一个芯片中，而 HOST 主桥在北桥中。但是从处理器体系结构的角度看，这些集成方式并不重要。

PCI 设备通过 HOST 主桥访问主存储器时，需要与处理器的 Cache 进行一致性操作，因此在设计 HOST 主桥时需要重点考虑 Cache 一致性操作。在 HOST 主桥中，还含有许多数据缓冲，以支持 PCI 总线的预读机制。

HOST 主桥是联系处理器与 PCI 设备的桥梁。在一个处理器系统中，每一个 HOST 主桥都管理了一棵 PCI 总线树，在同一棵 PCI 总线树上的所有 PCI 设备属于同一个 PCI 总线域。如图 1-1 所示，HOST 主桥 x 之下的 PCI 设备属于 PCI 总线 x 域，而 HOST 主桥 y 之下的 PCI 设备属于 PCI 总线 y 域。在这棵总线树上的所有 PCI 设备的配置空间都由 HOST 主桥通过配置读写总线周期访问。

如果 HOST 主桥支持 PCI V3.0 规范的 Peer-to-Peer 数据传送方式，那么分属不同 PCI 总线域的 PCI 设备可以直接进行数据交换。如图 1-1 所示，如果 HOST 主桥 y 支持 Peer-to-Peer 数据传送方式，PCI 设备 y01 可以直接访问 PCI 设备 01 或者 PCI 设备 11，而不需要通过处理器的参与。但是这种跨越总线域的数据传送方式在 PC 架构中并不常用，在 PC 架构中，重点考虑的是 PCI 设备与主存储器之间的数据交换，而不是 PCI 设备之间的数据交换。此外在 PC 架构中，具有两个 HOST 主桥的处理器系统也并不多见。

在 PowerPC 处理器中，HOST 主桥可以通过设置 Inbound 寄存器，使得分属于不同 PCI 总线域的设备可以直接通信。许多 PowerPC 处理器都具有多个 HOST 主桥，有关 PowerPC 处理器使用的 HOST 主桥详见第 2.2 节。

1.1.2 PCI 总线

在处理器系统中，含有 PCI 总线和 PCI 总线树这两个概念。这两个概念并不相同，在一棵 PCI 总线树中可能具有多条 PCI 总线，而具有血缘关系的 PCI 总线组成一棵 PCI 总线树。如在图 1-1 所示的处理器系统中，PCI 总线 x 树具有两条 PCI 总线，分别为 PCI 总线 x0 和 PCI 总线 x1。而 PCI 总线 y 树中仅有一条 PCI 总线。

PCI 总线由 HOST 主桥或者 PCI 桥管理，用来连接各类设备，如声卡、网卡和 IDE 接口卡等。在一个处理器系统中，可以通过 PCI 桥扩展 PCI 总线，并形成具有血缘关系的多级 PCI 总线，从而形成 PCI 总线树型结构。在处理器系统中有几个 HOST 主桥，就有几棵这样的 PCI 总线树，而每一棵 PCI 总线树都与一个 PCI 总线域对应。

与 HOST 主桥直接连接的 PCI 总线通常被命名为 PCI 总线 0。考虑到在一个处理器系统中可能有多个主桥，图 1-1 将 HOST 主桥 x 推出的 PCI 总线命名为 x0 总线，而将 PCI 桥 x1 扩展出的 PCI 总线称为 x1 总线，将 HOST 主桥 y 推出的 PCI 总线称为 y0~yn。分属不同 PCI 总线树的设备，其使用的 PCI 总线地址空间分属不同的 PCI 总线域空间。

1.1.3 PCI 设备

在 PCI 总线中有三类设备：PCI 主设备、PCI 从设备和桥设备。其中 PCI 从设备只能被动地接收来自 HOST 主桥或者其他 PCI 设备的读写请求；而 PCI 主设备可以通过总线仲裁获得 PCI 总线的使用权，主动地向其他 PCI 设备或者主存储器发起存储器读写请求。而桥设备的主要作用是管理下游的 PCI 总线，并转发上下游总线之间的总线事务。

一个 PCI 设备可以既是主设备也是从设备，但是在同一个时刻，这个 PCI 设备或者为主设备或者为从设备。PCI 总线规范将 PCI 主从设备统称为 PCI Agent 设备。在处理器系统中常见的 PCI 网卡、显卡、声卡等设备都属于 PCI Agent 设备。

在 PCI 总线中，HOST 主桥是一个特殊的 PCI 设备，该设备可以获取 PCI 总线的控制权访问 PCI 设备，也可以被 PCI 设备访问。但是 HOST 主桥并不是 PCI 设备。PCI 规范也没有规定如何设计 HOST 主桥。

在 PCI 总线中，还有一类特殊的设备，即桥设备。它包括 PCI 桥、PCI-to-(E)ISA 桥和 PCI-to-Cardbus 桥。本书重点介绍 PCI 桥，而不介绍其他桥设备的实现原理。PCI 桥的存在使 PCI 总线极具扩展性，处理器系统可以使用 PCI 桥进一步扩展 PCI 总线。

PCI 桥的出现使得采用 PCI 总线进行大规模系统互连成为可能。但是在目前已经实现的大规模处理器系统中，并没有使用 PCI 总线进行处理器系统与处理器系统之间的大规模互连。因为 PCI 总线是一个以 HOST 主桥为根的树型结构，使用主从架构，因而不易实现多处理器系统间的对等互连。

即便如此 PCI 桥仍然是 PCI 总线规范的精华所在，掌握 PCI 桥是深入理解 PCI 体系结构的基础。PCI 桥可以连接两条 PCI 总线，上游 PCI 总线和下游 PCI 总线，这两个 PCI 总线属于同一个 PCI 总线域，使用 PCI 桥扩展的所有 PCI 总线都同属于一个 PCI 总线域。

其中对 PCI 设备配置空间的访问仅可以从上游总线转发到下游总线，而数据传送可以双向进行。在 PCI 总线中，还存在一种非透明 PCI 桥，该桥片不是 PCI 总线规范定义的标准

桥片，但是适用于某些特殊应用，在第 2.5 节中将详细介绍这种桥片。在本书中，如不特别强调，PCI 桥是指透明桥，透明桥也是 PCI 总线规范定义的标准桥片。

PCI-to-(E)ISA 桥和 PCI-to-Cardbus 桥的主要作用是通过 PCI 总线扩展 (E)ISA 和 Cardbus 总线。在 PCI 总线推出之后，(E)ISA 总线并没有在处理器系统中立即消失，此时需要使用 PCI-(E)ISA 桥扩展 (E)ISA 总线，而使用 PCI-to-Cardbus 桥用来扩展 Cardbus 总线。本书并不关心 (E)ISA 和 Cardbus 总线的设计与实现。

1.1.4 HOST 处理器

PCI 总线规定在同一时刻内，在一棵 PCI 总线树上有且只有一个 HOST 处理器。这个 HOST 处理器可以通过 HOST 主桥，发起 PCI 总线的配置请求总线事务，并对 PCI 总线上的设备和桥片进行配置。

在 PCI 总线中，HOST 处理器是一个较为模糊的概念。在 SMP (Symmetric Multiprocessing) 处理器系统中，所有 CPU 都可以通过 HOST 主桥访问其下的 PCI 总线树，这些 CPU 都可以作为 HOST 处理器。但是值得注意的是，PCI 总线树的实际管理者是 HOST 主桥，而不是 HOST 处理器。

在 HOST 主桥中，设置了许多寄存器，HOST 处理器通过操作这些寄存器来管理 PCI 设备。如在 x86 处理器的 HOST 主桥中设置了 0xCF8 和 0xCFC 这两个 I/O 端口访问 PCI 设备的配置空间，而 PowerPC 处理器的 HOST 主桥设置了 CFG_ADDR 和 CFG_DATA 寄存器访问 PCI 设备的配置空间。值得注意的是，在 PowerPC 处理器中并没有 I/O 端口，因此使用存储器映像寻址方式访问外部设备的寄存器空间。

1.1.5 PCI 总线的负载

PCI 总线能挂接的负载与总线频率相关，其中总线频率越高，能挂接的负载越少。下面以 PCI 总线和 PCI-X 总线为例说明总线频率、峰值带宽和负载能力之间的关系，如表 1-1 所示。

表 1-1 PCI 总线频率、峰值带宽与负载能力之间的关系

总线类型	总线频率/MHz	峰值带宽/(MB/s)	负载能力
PCI	33	133	4~5 个插槽
	66	266	1~2 个插槽
PCI-X	66	266	4 个插槽
	133	533	2 个插槽
	266	1066	1 个插槽
	533	2131	1 个插槽

由表 1-1 可知，PCI 总线频率越高，能挂接的负载越少，但是整条总线能提供的带宽越大。值得注意的是，PCI-X 总线与 PCI 总线的传送协议略有不同，因此 66 MHz 的 PCI-X 总线的负载数较大。PCI-X 总线的详细说明见第 1.5 节。当 PCI-X 总线频率为 266 MHz 和 533 MHz 时，该总线只能挂接一个 PCI-X 插槽。在 PCI 总线中，一个插槽相当于两个负载，接插件和插卡各算为一个负载。在表 1-1 中，33 MHz 的 PCI 总线可以挂接 4~5 个插槽，相当于

直接挂接 8~10 个负载。

1.2 PCI 总线的信号定义

PCI 总线是一条共享总线，在一条 PCI 总线上可以挂接多个 PCI 设备。这些 PCI 设备通过一系列信号与 PCI 总线相连，这些信号由地址/数据信号、控制信号、仲裁信号、中断信号等多种信号组成。

PCI 总线是一个同步总线，每一个设备都具有一个 CLK 信号，其发送设备与接收设备使用这个 CLK 信号进行同步数据传递。PCI 总线可以使用 33 MHz 或者 66 MHz 的时钟频率，而 PCI-X 总线可以使用 133 MHz、266 MHz 或者 533 MHz 的时钟频率。

除了 RST#、INTA~D#、PME#和 CLKRUN#等信号之外，PCI 设备使用的绝大多数信号都使用这个 CLK 信号进行同步。其中 RST#是复位信号，而 PCI 设备使用 INTA~D#信号进行中断请求。本书并不详细介绍 PME#和 CLKRUN#信号。

1.2.1 地址和数据信号

在 PCI 总线中，与地址和数据相关的信号如下所示。

(1) AD[31:0] 信号

PCI 总线复用地址与数据信号。PCI 总线事务在启动后的第一个时钟周期传送地址，这个地址是 PCI 总线域的存储器地址或者 I/O 地址；而在下一个时钟周期传送数据^①。传送地址的时钟周期也被称为地址周期，而传送数据的时钟周期也被称为数据周期。PCI 总线支持突发传送，即在一个地址周期之后，可以紧跟多个数据周期。

(2) PAR 信号

PCI 总线使用奇偶校验机制，保证地址和数据信号在进行数据传递时的正确性。PAR 信号是 AD[31:0]和 C/BE[3:0]的奇偶校验信号。PCI 主设备在地址周期和数据周期中，使用该信号为地址和数据信号线提供奇偶校验位。

(3) C/BE[3:0]#信号

PCI 总线复用命令与字节选通引脚。在地址周期中，C/BE[3:0]#信号表示 PCI 总线的命令。而在数据周期中，C/BE[3:0]#引脚输出字节选通信号，其中 C/BE3#、C/BE2#、C/BE1# 和 C/BE0#与数据的字节 3、2、1 和 0 对应。使用这组信号可以对 PCI 设备进行单个字节、字和双字访问。PCI 总线通过 C/BE[3:0]#信号定义了多个总线事务，这些总线事务如表 1-2 所示。

表 1-2 PCI 总线事务

C/BE[3:0]#	命令类型	说 明
0000	Interrupt Acknowledge	中断响应总线事务读取当前挂接在 PCI 总线上的中断控制器的中断向量号。目前大多数处理器系统的中断控制器都不挂接在 PCI 总线上，因此这种总线事务很少使用

① 双地址周期在第一、二个时钟周期，都传送地址。

(续)

C/BE[3:0]#	命令类型	说明
0001	Special Cycle	HOST 主桥可以使用 Special Cycle 事务在 PCI 总线上进行信息广播
0010	I/O Read	HOST 主桥可以使用该总线事务对 PCI 设备的 I/O 地址空间进行读操作。目前多数 PCI 设备都不支持 I/O 地址空间，而仅支持存储器地址空间，但是仍有部分 PCI 设备同时包含 I/O 地址空间和存储器地址空间
0011	I/O Write	对 PCI 总线的 I/O 地址空间进行写操作
0100	Reserved	保留
0101	Reserved	保留
0110	Memory Read	HOST 主桥可以使用该总线事务对 PCI 设备的存储器空间进行读操作。PCI 设备也可以使用该总线事务读取处理器的存储器空间
0111	Memory Write	HOST 主桥可以使用该总线事务对 PCI 设备的存储器空间进行写操作。PCI 设备也可以使用该总线事务向处理器的存储器空间进行写操作
1000	Reserved	保留
1001	Reserved	保留
1010	Configuration Read	HOST 主桥可以对 PCI 设备的配置空间进行读操作。每一个 PCI 设备都有独立的配置空间。在多功能 PCI 设备中，每一个子设备（Function）也有一个独立的配置空间。该总线事务只能由 HOST 主桥发出，PCI 桥可以转发该总线事务
1011	Configuration Write	HOST 主桥对 PCI 设备的配置空间进行写操作
1100	Memory Read Multiple	HOST 主桥可以使用该总线事务对 PCI 设备的存储器空间进行多行读操作，这种操作并不多见。该总线事务的主要用途是供 PCI 设备使用，读取主存储器。这个读操作与 Memory Read 操作（C/BE [3: 0] 为 0x0110 时）略有不同，详见第 3.4.5 节
1101	Dual Address Cycle	PCI 总线支持 64 位地址，处理器或者其他 PCI 设备访问 64 位 PCI 总线地址时，必须使用双地址周期产生 64 位的 PCI 总线地址。PCI 设备使用 DMA 读写方式访问 64 位的存储器地址时，也可以使用该总线事务
1110	Memory Read Line	HOST 主桥可以使用该总线事务对 PCI 设备的存储器空间进行单行读操作，这种操作并不多见。该总线事务的主要用途是供 PCI 设备使用，读取主存储器。详见第 3.4.5 节
1111	Memory Write and Invalidate	存储器写并无效操作，与存储器写不同，PCI 设备可以使用该总线事务对主存储器空间进行写操作。该总线事务将数据写入主存储器的同时，将对应 Cache 行中的数据“使无效”，详见第 3.3.4 节

1.2.2 接口控制信号

在 PCI 总线中，接口控制信号的主要作用是保证数据的正常传递，并根据 PCI 主从设备的状态，暂停、终止或者正常完成当前总线事务，其主要信号如下。

(1) FRAME#信号

该信号指示一个 PCI 总线事务的开始与结束。当 PCI 设备获得总线的使用权后，将置该信号有效，即置为低，启动 PCI 总线事务，当结束总线事务时，将置该信号无效，即置为高。PCI 设备（包括 HOST 主桥）只有通过仲裁获得当前 PCI 总线的使用权后，才能驱动该信号。

(2) IRDY#信号

该信号由 PCI 主设备（包括 HOST 主桥）驱动，该信号有效时表示 PCI 主设备的数据已经准备完毕。如果当前 PCI 总线事务为写事务，表示数据已经在 AD[31:0]上有效；如果为读事务，表示 PCI 目标设备已经准备好接收缓冲，目标设备可以将数据发送到 AD[31:0]上。

(3) TRDY#信号

该信号由目标设备驱动，该信号有效时表示目标设备已经将数据准备完毕。如果当前 PCI 总线事务为写事务，表示目标设备已经准备好接收缓冲，可以将 AD[31:0]上的数据写入目标设备；如果为读事务，表示 PCI 设备需要的数据已经在 AD[31:0]上有效。

该信号可以和 IRDY#信号联合使用，在 PCI 总线事务上插入等待周期，对 PCI 总线的数据传送进行控制。

(4) STOP#信号

该信号有效时表示目标设备请求主设备停止当前 PCI 总线事务。一个 PCI 总线事务除了可以正常结束外，目标设备还可以使用该信号终止当前 PCI 总线事务。目标设备可以根据不同的情况，要求主设备对当前 PCI 总线事务进行重试（Retry）、断连（Disconnect），也可以向主设备报告目标设备夭折（Target Abort）。

目标设备要求主设备 Retry 和 Disconnect 并不意味着当前 PCI 总线事务出现错误。当目标设备没有将数据准备好时，可以使用 Retry 周期使主设备重试当前 PCI 总线事务。有时目标设备不能接收来自主设备较长的 Burst 操作时，可以使用 Disconnect 周期，将一个较长的 Burst 操作分解为多个 Burst 操作。当主设备访问的地址越界时，目标设备可以使用 Disconnect 周期，终止主设备的越界访问。

而 Target Abort 表示在数据传送中出现错误。处理器系统必须对这种情况进行处理。在 PCI 总线中，出现 Abort 一般意味着当前 PCI 总线域出现了较为严重的错误。

(5) IDSEL 信号

PCI 总线在进行配置读写总线事务时，使用该信号选择 PCI 目标设备。配置读写总线事务与存储器读写总线事务在实现上略有不同。在 PCI 总线中，存储器读写总线事务使用地址译码方式访问外部设备。而配置读写总线事务使用“ID 译码方式”访问 PCI 设备，即通过 PCI 设备的总线号、设备号和寄存器号访问 PCI 设备的配置空间。

IDSEL 信号与 PCI 设备的设备号相关，相当于 PCI 设备配置空间的片选信号，这部分内容将在第 2.4.4 节中详细介绍。

(6) DEVSEL#信号

该信号有效时表示 PCI 总线的目标设备准备好，该信号与 TRDY#信号的不同之处在于该信号有效仅表示目标设备已经完成了地址译码。目标设备使用该信号通知 PCI 主设备，其访问对象在当前 PCI 总线上，但是并不表示目标设备可以与主设备进行数据交换。而 TRDY#信号表示数据有效，PCI 主设备可以向目标设备写入或者从目标设备读取数据。

PCI 总线规范根据设备的译码速度，将 PCI 设备分为快速、中速和慢速三种。在 PCI 总线上还有一种特殊的设备，即负向译码设备，在一条 PCI 总线上当快速、中速和慢速三种设备都不能响应 PCI 总线事务的地址时，负向译码设备将被动地接收这个 PCI 总线事务。如果在 PCI 主设备访问的 PCI 总线上，没有任何设备可以置 DEVSEL# 信号为有效，主设备将使用 Master Abort 周期结束当前总线事务。

(7) LOCK# 信号

PCI 主设备可以使用该信号，将目标设备的某个存储器或者 I/O 资源锁定，以禁止其他 PCI 主设备访问此资源，直到锁定这个资源的主设备将其释放。PCI 总线使用 LOCK# 信号实现 LOCK 总线事务，只有 HOST 主桥、PCI 桥或者其他桥片可以使用 LOCK# 信号。在 PCI 总线的早期版本中，PCI Agent 设备也可以使用 LOCK# 信号，而目前 PCI 总线使用 LOCK# 信号仅是为防止死锁和向前兼容。LOCK 总线事务将严重影响 PCI 总线的传送效率，在实际应用中，设计者应当尽量避免使用该总线事务。

1.2.3 仲裁信号

PCI 设备使用该组信号进行总线仲裁，并获得 PCI 总线的使用权。只有 PCI 主设备需要使用该组信号，而 PCI 从设备可以不使用总线仲裁信号。这组信号由 REQ# 和 GNT# 组成。其中 PCI 主设备的 REQ# 和 GNT# 信号与 PCI 总线的仲裁器直接相连。

PCI 主设备的总线仲裁信号与 PCI 总线仲裁器的连接关系如图 1-2 所示。值得注意的是，每一个 PCI 主设备都具有独立的总线仲裁信号，并与 PCI 总线仲裁器一一相连。而总线仲裁器需要保证在同一个时间段内，只有一个 PCI 设备可以使用当前总线。

在一个处理器系统中，一条 PCI 总线可以挂接 PCI 主设备的数目，除了与负载能力相关之外，还与 PCI 总线仲裁器能够提供的仲裁信号数目直接相关。

在一棵 PCI 总线树中，每一条 PCI 总线上都有一个总线仲裁器。一个处理器系统可以使用 PCI 桥扩展出一条新的 PCI 总线，这条新的 PCI 总线也需要一个总线仲裁器，通常在 PCI 桥中集成了这个总线仲裁器。多数 HOST 主桥也集成了一个 PCI 总线仲裁器，但是 PCI 总线也可以使用独立的 PCI 总线仲裁器。

PCI 主设备使用 PCI 总线进行数据传递时，需要首先置 REQ# 信号有效，向 PCI 总线仲裁器发出总线申请，当 PCI 总线仲裁器允许 PCI 主设备获得 PCI 总线的使用权后，将置 GNT# 信号为有效，并将其发送给指定的 PCI 主设备。而 PCI 主设备在获得总线使用权之后，将可以置 FRAME# 信号有效，与 PCI 从设备进行数据通信。

1.2.4 中断请求等其他信号

PCI 总线提供了 INTA#、INTB#、INTC# 和 INTD# 四个中断请求信号，PCI 设备借助这些中断请求信号，使用电平触发方式向处理器提交中断请求。当这些中断请求信号为低时，PCI 设备将向处理器提交中断请求；当处理器执行中断服务程序清除 PCI 设备的中断请求

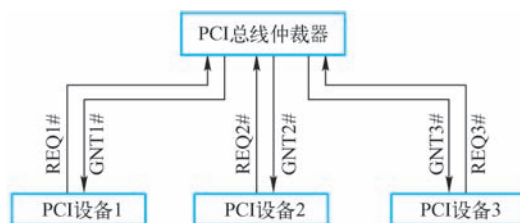


图 1-2 PCI 设备与总线仲裁器的连接关系

后，PCI 设备将该信号置高[⊖]，结束当前中断请求。

PCI 总线规定单功能设备只能使用 INTA#信号，而多功能设备才能使用 INTB#/C#/D#信号。PCI 设备的这些中断请求信号可以通过某种规则进行线与，之后与中断控制器的中断请求信号线相连。而处理器系统需要预先知道这个规则，以便正确处理来自不同 PCI 设备的中断请求，这个规则也被称为中断路由表，有关中断路由表的详细描述见第 1.4.2 节。

PCI 总线在进行数据传递过程时，难免会出现各种各样的错误，因此 PCI 总线提供了一些错误信号，如 PERR#和 SERR#信号。其中当 PERR#信号有效时，表示数据传送过程中出现奇偶校验错（Special Cycle 周期除外）；而当 SERR#信号有效时，表示当前处理器系统出现了三种错误可能，分别为地址奇偶校验错、在 Special Cycle 周期中出现数据奇偶校验错、系统出现其他严重错误。

如果 PCI 总线支持 64 位模式，还需要提供 AD[63:32]、C/BE[7:4]、REQ64、ACK64 和 PAR64 这些信号。此外 PCI 总线还有一些与 JTAG、SMBCLK 以及 66MHz 使能相关的信号，本章并不介绍这些信号。

1.3 PCI 总线的存储器读写总线事务

总线的基本任务是实现数据传送，将一组数据从一个设备传送到另一个设备，当然总线也可以将一个设备的数据广播到多个设备。在处理器系统中，这些数据传送都要依赖一定的规则，PCI 总线并不例外。

PCI 总线使用单端并行数据线，采用地址译码方式进行数据传递，而采用 ID 译码方式进行配置信息的传递。其中地址译码方式使用地址信号，而 ID 译码方式使用 PCI 设备的 ID 号，包括 Bus Number、Device Number、Function Number 和 Register Number。下面将以图 1-1 中的处理器系统为例，简要介绍 PCI 总线支持的总线事务及其传送方式。

由表 1-2 可知，PCI 总线支持多种总线事务。本节重点介绍存储器读写总线事务与 I/O 读写总线事务，并在第 2.4 节详细介绍配置读写总线事务。值得注意的是，PCI 设备只有在系统软件初始化配置空间之后，才能够被其他主设备访问。

当 PCI 设备的配置空间被初始化之后，该设备在当前的 PCI 总线树上将拥有一个独立的 PCI 总线地址空间，即 BAR（Base Address Register）寄存器所描述的空间，有关 BAR 寄存器的详细说明见第 2.3.2 节。

处理器与 PCI 设备进行数据交换，或者 PCI 设备之间进行存储器数据交换时，都将通过 PCI 总线地址完成。而 PCI 设备与主存储器进行 DMA 操作时，使用的也是 PCI 总线域的地址，而不是存储器域的地址，此时 HOST 主桥将完成 PCI 总线地址到存储器域地址的转换，不同的 HOST 主桥进行地址转换时使用的方法并不相同。

PCI 总线的配置读写总线事务与 HOST 主桥与 PCI 桥相关，因此读者需要了解 HOST 主桥和 PCI 桥的详细实现机制之后，才能深入理解这部分内容。在第 2.4 节将详细介绍这些内容。在下文中，假定所使用的 PCI 设备的配置空间已经被系统软件初始化。

⊖ INTx#这组信号为开漏输出，当所有的驱动源不驱动该信号时，该信号由上拉电阻驱动为高。

PCI 总线支持以下几类存储器读写总线事务。

1) HOST 处理器对 PCI 设备的 BAR 空间进行数据读写，BAR 空间可以使用存储器或者 I/O 译码方式。HOST 处理器使用 PCI 总线的存储器读写总线事务和 I/O 读写总线事务访问 PCI 设备的 BAR 空间。

2) PCI 设备之间的数据传递。在 PCI 总线上的两个设备可以直接通信，如一个 PCI 设备可以访问另外一个设备的 BAR 空间。不过这种数据传递在 PC 处理器系统中较少使用。

3) PCI 设备对主存储器进行读写，即 DMA 读写操作。DMA 读写操作在所有处理器系统中都较为常用，也是 PCI 总线数据传送的重点。在多数情况下，DMA 读写操作结束后将伴随着中断的产生。PCI 设备可以使用 INTA#、INTB#、INTC#和 INTD#信号提交中断请求，也可以使用 MSI 机制提交中断请求。

1.3.1 PCI 总线事务的时序

PCI 总线使用第 1.2 节所述的信号进行数据和配置信息的传递，一个 PCI 总线事务的基本访问时序如图 1-3 所示，与 PCI 总线事务相关的控制信号有 FRAME#、IRDY#、TRDY#、DEVSEL#等其他信号。

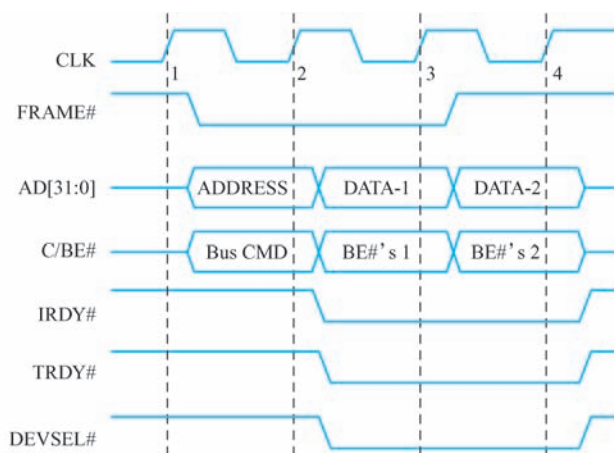


图 1-3 PCI 总线事务的基本访问时序

当一个 PCI 主设备需要使用 PCI 总线时，首先需要发送 REQ#信号，通过总线仲裁获得总线使用权，即 GNT#信号有效后，使用以下步骤完成一个完整 PCI 总线事务，对目标设备进行存储器或者 I/O 地址空间的读写访问。

1) 当 PCI 主设备获得总线使用权之后，将在 CLK1 的上升沿置 FRAME#信号有效，启动 PCI 总线事务。当 PCI 总线事务结束后，FRAME#信号将被置为无效。

2) PCI 总线周期的第一个时钟周期（CLK1 的上升沿到 CLK2 的上升沿之间）为地址周期。在地址周期中，PCI 主设备将访问的目的地址和总线命令分别驱动到 AD[31:0]和 C/BE#信号上。如果当前总线命令是配置读写，那么 IDSEL 信号线也被置为有效，IDSEL 信号与 PCI 总线的 AD[31:11]相连，详见第 2.4.4 节。

3) 当 IRDY#、TRDY#和 DEVSEL#信号都有效后，总线事务将使用数据周期进行数据传递。当 IRDY#和 TRDY#信号没有同时有效时，PCI 总线不能进行数据传递，PCI 总线使用这

两个信号进行传送控制。

4) PCI 总线支持突发周期, 因此在地址周期之后可以有多个数据周期, 可以传送多组数据。而目标设备并不知道突发周期的长度, 如果目标设备不能继续接收数据时, 可以 disconnect (断连) 当前总线事务。值得注意的是, 只有存储器读写总线事务可以使用突发周期。

一个完整的 PCI 总线事务远比上述过程复杂得多, 因为 PCI 总线还支持许多传送方式, 如双地址周期、fast back-to-back (快速背靠背)、插入等待状态、重试和断连、总线上的错误处理等一系列总线事务。本书不一一介绍这些传送方式。

1.3.2 Posted 和 Non-Posted 传送方式

PCI 总线规定了两类数据传送方式, 分别是 Posted 和 Non-Posted 数据传送方式。其中使用 Posted 数据传送方式的总线事务也被称为 Posted 总线事务; 而使用 Non-Posted 数据传送方式的总线事务也被称为 Non-Posted 总线事务。

其中 Posted 总线事务指 PCI 主设备向 PCI 目标设备进行数据传递时, 当数据到达 PCI 桥后, 即由 PCI 桥接管来自上游总线的总线事务, 并将其转发到下游总线。采用这种数据传送方式, 在数据还没有到达最终的目的地之前, PCI 总线就可以结束当前总线事务, 从而在一定程度上解决了 PCI 总线的拥塞问题。

而 Non-Posted 总线事务是指 PCI 主设备向 PCI 目标设备进行数据传递时, 数据必须到达最终目的地之后, 才能结束当前总线事务的一种数据传递方式。

显然采用 Posted 传送方式, 当这个 Posted 总线事务通过某条 PCI 总线后, 就可以释放 PCI 总线的资源; 而采用 Non-Posted 传送方式, PCI 总线在没有结束当前总线事务时必须等待。这种等待将严重阻塞当前 PCI 总线上的其他数据传送, 因此 PCI 总线使用 Delayed 总线事务处理 Non-Posted 数据请求, 使用 Delayed 总线事务可以相对缓解 PCI 总线的拥塞。Delayed 总线事务的详细介绍见第 1.3.5 节。

PCI 总线规定只有存储器写请求 (包括存储器写并无效请求) 可以采用 Posted 总线事务, 下文将 Posted 存储器写请求简称为 PMW (Posted Memory Write), 而存储器读请求、I/O 读写请求、配置读写请求只能采用 Non-Posted 总线事务。

下面以图 1-1 的处理器系统中的 PCI 设备 11 向存储器进行 DMA 写操作为例, 说明 Posted 传送方式的实现过程。PCI 设备 11 进行 DMA 写操作时使用存储器写总线事务, 当 PCI 设备 11 获得 PCI 总线 x1 的使用权后, 将发送存储器写总线事务到 PCI 总线 x1。当 PCI 桥 x1 发现这个总线事务的地址不在该桥管理的地址范围内将首先接收这个总线事务, 并结束 PCI 总线 x1 的总线事务。

此时 PCI 总线 x1 使用的资源已被释放, PCI 设备 11 和 PCI 设备 12 可以使用 PCI 总线 x1 进行通信。PCI 桥 x1 获得 PCI 总线 x0 的使用权后, 将转发这个存储器写总线事务到 PCI 总线 x0, 之后 HOST 主桥 x 将接收这个存储器写总线事务, 并最终将数据写入主存储器。

由以上过程可以发现, Posted 数据请求在通过 PCI 总线之后, 将逐级释放总线资源, 因此 PCI 总线的利用率较高。而使用 Non-Posted 方式进行数据传送的处理过程与此不同, Non-Posted 数据请求在通过 PCI 总线时, 并不会及时释放总线资源, 从而在某种程度上影响 PCI

总线的使用效率和传送带宽。

1.3.3 HOST 处理器访问 PCI 设备

HOST 处理器对 PCI 设备的数据访问主要包含两方面内容，一方面是处理器向 PCI 设备发起存储器和 I/O 读写请求；另一方面是处理器对 PCI 设备进行配置读写。

在 PCI 设备的配置空间中，共有 6 个 BAR 寄存器。每一个 BAR 寄存器都与 PCI 设备使用的一组 PCI 总线地址空间对应，BAR 寄存器记录这组地址空间的基地址。本书将与 BAR 寄存器对应的 PCI 总线地址空间称为 BAR 空间，在 BAR 空间中可以存放 I/O 地址空间，也可以存放存储器地址空间。

PCI 设备可以根据需要，有选择地使用这些 BAR 空间。值得注意的是，在 BAR 寄存器中存放的是 PCI 设备使用的“PCI 总线域”的物理地址，而不是“存储器域”的物理地址，有关 BAR 寄存器的详细介绍见第 2.3.2 节。

HOST 处理器访问 PCI 设备 I/O 地址空间的过程，与访问存储器地址空间略有不同。有些处理器，如 x86 处理器，具有独立的 I/O 地址空间。x86 处理器可以将 PCI 设备使用的 I/O 地址映射到存储器域的 I/O 地址空间中，之后处理器可以使用 IN、OUT 等指令对存储器域的 I/O 地址进行访问，然后通过 HOST 主桥将存储器域的 I/O 地址转换为 PCI 总线域的 I/O 地址，最后使用 PCI 总线的 I/O 总线事务对 PCI 设备的 I/O 地址进行读写访问。在 x86 处理器中，存储器域的 I/O 地址与 PCI 总线域的 I/O 地址相同。

对于有些没有独立 I/O 地址空间的处理器，如 PowerPC 处理器，需要在 HOST 主桥初始化时，将 PCI 设备使用的 I/O 地址空间映射为处理器的存储器地址空间。PowerPC 处理器对这段“存储器域”的存储器空间进行读写访问时，HOST 主桥将存储器域的这段存储器地址转换为 PCI 总线域的 I/O 地址，然后通过 PCI 总线的 I/O 总线事务对 PCI 设备的 I/O 地址进行读写操作。

在 PCI 总线中，存储器读写事务与 I/O 读写事务的实现较为类似。首先 HOST 处理器在初始化时，需要将 PCI 设备使用的 BAR 空间映射到“存储器域”的存储器地址空间。之后处理器通过存储器读写指令访问“存储器域”的存储器地址空间，HOST 主桥将“存储器域”的读写请求翻译为 PCI 总线的存储器读写总线事务之后，再发送给目标设备。

值得注意的是存储器域和 PCI 总线域的概念。PCI 设备能够直接使用的地址是 PCI 总线域的地址，在 PCI 总线事务中出现的地址也是 PCI 总线域的地址；而处理器能够直接使用的地址是存储器域的地址。理解存储器域与 PCI 总线域的区别对于理解 PCI 总线至关重要，在第 2.1 节将专门讨论这两个概念。

以上对 PCI 总线的存储器与 I/O 总线事务的介绍并没有考虑 PCI 桥的存在，如果将 PCI 桥考虑进来，情况将略微复杂。下面将以图 1-1 为例说明处理器如何通过 HOST 主桥和 PCI 桥 x1 对 PCI 设备 11 进行存储器读写操作。当处理器对 PCI 设备 11 进行存储器写操作时，这些数据需要通过 HOST 主桥 x 和 PCI 桥 x1，最终到达 PCI 设备 11，其访问步骤如下。值得注意的是，以下步骤忽略 PCI 总线的仲裁过程。

1) 首先处理器将要传递的数据放入通用寄存器中，之后向 PCI 设备 11 映射到的存储器域的地址进行写操作。值得注意的是，处理器并不能直接访问 PCI 设备 11 的 PCI 总线地址空间，因为这些地址空间是属于 PCI 总线域的，处理器所能直接访问的空间是存储器域的地

址空间。处理器必须通过 HOST 主桥将存储器域的数据访问转换为 PCI 总线事务才能对 PCI 总线地址空间进行访问。

2) HOST 主桥 x 接收来自处理器的存储器写请求, 之后处理器结束当前存储器写操作, 释放系统总线。HOST 主桥 x 将存储器域的存储器地址转换为 PCI 总线域的 PCI 总线地址。并向 PCI 总线 x0 发起 PCI 写请求总线事务。值得注意的是, 虽然在许多处理器系统中, 存储器地址和 PCI 总线地址完全相等, 但其含义并不相同。

3) PCI 总线 x0 上的 PCI 设备 01、PCI 设备 02 和 PCI 桥 x1 将同时监听这个 PCI 写总线事务。最后 PCI 桥 x1 接收这个写总线事务, 并结束来自 PCI 总线 x0 的 PCI 总线事务。之后 PCI 桥 x1 向 PCI 总线 x1 发起新的 PCI 总线写总线事务。

4) PCI 总线 x1 上的 PCI 设备 11 和 PCI 设备 12 同时监听这个 PCI 写总线事务。最后 PCI 设备 11 通过地址译码方式接收这个写总线事务, 并结束来自 PCI 总线 x1 上的 PCI 总线事务。

由以上过程可以发现, 由于存储器写总线事务使用 Posted 传送方式, 因此数据通过 PCI 桥后都将结束上一级总线的 PCI 总线事务, 从而上一级 PCI 总线可以被其他 PCI 设备使用。如果使用 Non-Posted 传送方式, 直到数据发送到 PCI 设备 11 之后, PCI 总线 x1 和 x0 才能依次释放, 从而在某种程度上将造成 PCI 总线的拥塞。

处理器对 PCI 设备 11 进行 I/O 读写操作和存储器读操作时只能采用 Non-Posted 方式进行, 与 Posted 方式相比, 使用 Non-Posted 方式, 当数据到达目标设备后, 目标设备需要向主设备发出“回应[⊖]”, 当主设备收到这个“回应”后才能结束整个总线事务。本节不再讲述处理器如何对 PCI 设备进行 I/O 写操作, 请读者思考这个过程。

处理器对 PCI 设备 11 进行存储器读时, 这个读请求需要首先通过 HOST 主桥 x 和 PCI 桥 x1 到达 PCI 设备, 之后 PCI 设备将读取的数据再次通过 PCI 桥 x1 和 HOST 主桥 x 传递给 HOST 处理器, 其步骤如下所示。我们首先假设 PCI 总线没有使用 Delayed 传送方式处理 Non-Posted 总线事务, 而是使用纯粹的 Non-Posted 方式。

1) 首先处理器准备接收数据使用的通用寄存器, 之后向 PCI 设备 11 映射到的存储器域的地址进行读操作。

2) HOST 主桥 x 接收来自处理器的存储器读请求。HOST 主桥 x 进行存储器地址到 PCI 总线地址的转换, 之后向 PCI 总线 x0 发起存储器读总线事务。

3) PCI 总线 x0 上的 PCI 设备 01、PCI 设备 02 和 PCI 桥 x1 将监听这个存储器读请求, 之后 PCI 桥 x1 接收这个存储器读请求。然后 PCI 桥 x1 向 PCI 总线 x1 发起新的 PCI 总线读请求。

4) PCI 总线 x1 上的 PCI 设备 11 和 PCI 设备 12 监听这个 PCI 读请求总线事务。最后 PCI 设备 11 接收这个存储器读请求总线事务, 并将这个读请求总线事务转换为存储器读完成总线事务之后, 将数据传送到 PCI 桥 x1, 并结束来自 PCI 总线 x1 上的 PCI 总线事务。

5) PCI 桥 x1 将接收到的数据通过 PCI 总线 x0, 继续上传到 HOST 主桥 x, 并结束 PCI 总线 x0 上的 PCI 总线事务。

⊖ 如果是存储器、I/O 读或者配置读总线事务, 这个回应包含数据; 如果是 I/O 写或者配置写, 这个回应不包含数据。

6) HOST 主桥 x 将数据传递给处理器，最终结束处理器的存储器读操作。

显然这种方式与 Posted 传送方式相比，PCI 总线的利用率较低。因为只要 HOST 处理器没有收到来自目标设备的“回应”，那么 HOST 处理器到目标设备的传送路径上使用的所有 PCI 总线都将被阻塞。因而 PCI 总线 x0 和 x1 并没有被充分利用。

由以上例子，可以发现只有“读完成”依次通过 PCI 总线 x1 和 x0 之后，存储器读总线事务才不继续占用 PCI 总线 x1 和 x0 的资源，显然这种数据传送方式并不合理。因此 PCI 总线使用 Delayed 传送方式解决这个总线拥塞问题，有关 Delayed 传送方式的实现机制见第 1.3.5 节。

1.3.4 PCI 设备读写主存储器

PCI 设备与存储器直接进行数据交换的过程也被称为 DMA。与其他总线的 DMA 过程类似，PCI 设备进行 DMA 操作时，需要获得数据传送的目的地址和传送大小。支持 DMA 传递的 PCI 设备可以在其 BAR 空间中设置两个寄存器，分别保存这个目标地址和传送大小。这两个寄存器也是 PCI 设备 DMA 控制器的组成部件。

值得注意的是，PCI 设备进行 DMA 操作时，使用的目的地址是 PCI 总线域的物理地址，而不是存储器域的物理地址，因为 PCI 设备并不能识别存储器域的物理地址，而仅能识别 PCI 总线域的物理地址。

HOST 主桥负责完成 PCI 总线地址到存储器域地址的转换。HOST 主桥需要进行合理设置，将存储器的地址空间映射到 PCI 总线之后，PCI 设备才能对这段存储器空间进行 DMA 操作。PCI 设备不能直接访问没有经过主桥映射的存储器空间。

许多处理器允许 PCI 设备访问所有存储器域地址空间，但是有些处理器可以设置 PCI 设备所能访问的存储器域地址空间，从而对存储器域地址空间进行保护。例如 PowerPC 处理器的 HOST 主桥可以使用 Inbound 寄存器组，设置 PCI 设备访问的存储器地址范围和属性，只有在 Inbound 寄存器组映射的存储器空间才能被 PCI 设备访问，在第 2.2 节将详细介绍 PowerPC 处理器的这组寄存器。

综上所述，在一个处理器系统中，并不是所有存储器空间都可以被 PCI 设备访问，只有在 PCI 总线域中有映像的存储器空间才能被 PCI 设备访问。经过 HOST 主桥映射的存储器，具有两个“地址”，一个是在存储器域的地址，一个是在 PCI 总线域的 PCI 总线地址。当处理器访问这段存储器空间时，使用存储器地址；而 PCI 设备访问这段内存时，使用 PCI 总线地址。在多数处理器系统中，存储器地址与 PCI 总线地址相同，但是系统程序员需要正确理解这两个地址的区别。

下面以 PCI 设备 11 向主存储器写数据为例，说明 PCI 设备如何进行 DMA 写操作。

1) 首先 PCI 设备 11 将存储器写请求发向 PCI 总线 x1，注意这个写请求使用的地址是 PCI 总线域的地址。

2) PCI 总线 x1 上的所有设备监听这个请求，因为 PCI 设备 11 是向处理器的存储器写数据，所以 PCI 总线 x1 上的 PCI Agent 设备都不会接收这个数据请求。

3) PCI 桥 x1 发现当前总线事务使用的 PCI 总线地址不是其下游设备使用的 PCI 总线地址，则接收这个数据请求，有关 PCI 桥的 Secondary 总线接收数据的过程见第 3.2.1 节。此时 PCI 桥 x1 将结束来自 PCI 设备 11 的 Posted 存储器写请求，并将这个数据请求推到上游

PCI 总线上，即 PCI 总线 x0 上。

4) PCI 总线 x0 上的所有 PCI 设备包括 HOST 主桥将监听这个请求。PCI 总线 x0 上的 PCI Agent 设备也不会接收这个数据请求，此时这个数据请求将由 HOST 主桥 x 接收，并结束 PCI 桥 x1 的 Posted 存储器写请求。

5) HOST 主桥 x 发现这个数据请求发向存储器，则将来自 PCI 总线 x0 的 PCI 总线地址转换为存储器地址，之后通过存储器控制器将数据写入存储器，完成 PCI 设备的 DMA 写操作。

PCI 设备进行 DMA 读过程与 DMA 写过程较为类似。不过 PCI 总线的存储器读总线事务只能使用 Non-Posted 总线事务，其过程如下。

1) 首先 PCI 设备 11 将存储器读请求发向 PCI 总线 x1。

2) PCI 总线 x1 上的所有设备监听这个请求，因为 PCI 设备 11 是从存储器中读取数据，所以 PCI 总线 x1 上的设备，如 PCI 设备 12，不会接收这个数据请求。PCI 桥 x1 发现下游 PCI 总线没有设备接收这个数据请求，则接收这个数据请求，并将这个数据请求推到上游 PCI 总线上，即 PCI 总线 x0 上。

3) PCI 总线 x0 上的设备将监听这个请求。PCI 总线 x0 上的设备也不会接收这个数据请求，最后这个数据请求将由 HOST 主桥 x 接收。

4) HOST 主桥 x 发现这个数据请求是发向主存储器的，则将来自 PCI 总线 x0 的 PCI 总线地址转换为存储器地址，之后通过存储器控制器将数据读出，并转发到 HOST 主桥 x。

5) HOST 主桥 x 将数据经由 PCI 桥 x1 传递到 PCI 设备 11，PCI 设备 11 接收到这个数据后结束 DMA 读。

以上过程仅是 PCI 设备向存储器读写数据的一个简单流程。如果考虑处理器中的 Cache，这些存储器读写过程较为复杂。

PCI 总线还允许 PCI 设备之间进行数据传递，PCI 设备间的数据交换较为简单。在实际应用中，PCI 设备间的数据交换并不常见。下面以图 1-1 为例，简要介绍 PCI 设备 11 将数据写入 PCI 设备 01 的过程；请读者自行考虑 PCI 设备 11 从 PCI 设备 01 读取数据的过程。

1) 首先 PCI 设备 11 将 PCI 写总线事务发向 PCI 总线 x1 上。PCI 桥 x1 和 PCI 设备 12 同时监听这个写总线事务。

2) PCI 桥 x1 将接收这个 PCI 写请求总线事务，并将这个 PCI 写总线事务上推到 PCI 总线 x0。

3) PCI 总线 x0 上的所有设备将监听这个 PCI 写总线事务，最后由 PCI 设备 01 接收这个数据请求，并完成 PCI 写事务。

1.3.5 Delayed 传送方式

如上所述，当处理器使用 Non-Posted 总线周期对 PCI 设备进行读操作，或者 PCI 设备使用 Non-Posted 总线事务对存储器进行读操作时，如果数据没有到达目的地，那么在这个读操作路径上的所有 PCI 总线都不能被释放，这将严重影响 PCI 总线的使用效率。

为此 PCI 桥需要对 Non-Posted 总线事务进行优化处理，并使用 Delayed 总线事务处理这些 Non-Posted 总线事务，PCI 总线规定只有 Non-Posted 总线事务可以使用 Delayed 总线事务。PCI 总线的 Delay 总线事务由 Delay 读写请求和 Delay 读写完成总线事务组成，当 Delay 读写

请求到达目的地后，将被转换为 Delay 读写完成总线事务。基于 Delay 总线请求的数据交换如图 1-4 所示。

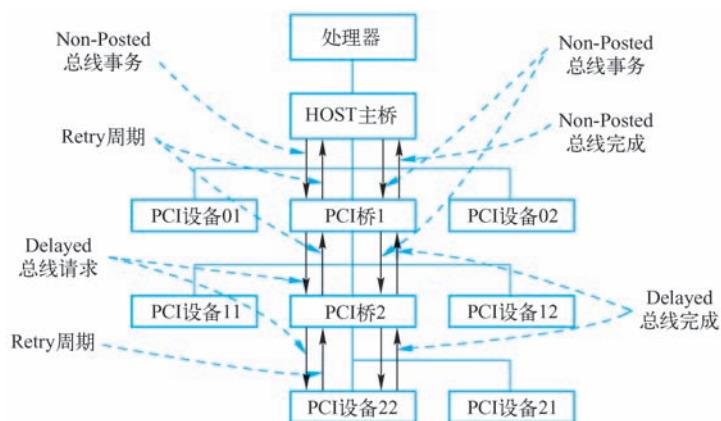


图 1-4 基于 Delayed 总线请求的数据交换

假设处理器通过存储器读、I/O 读写或者配置读写访问 PCI 设备 22 时，首先经过 HOST 主桥进行存储器域与 PCI 总线域的地址转换，并由 HOST 主桥发起 PCI 总线事务，然后通过 PCI 桥 1、2，最终到达 PCI 设备 22。其详细步骤如下。

1) HOST 主桥完成存储器域到 PCI 总线域的转换，然后启动 PCI 读总线事务。

2) PCI 桥 1 接收这个读总线事务，并首先使用 Retry 周期，使 HOST 主桥择时重新发起相同的总线周期。此时 PCI 桥 1 的上游 PCI 总线将被释放。值得注意的是 PCI 桥并不会每一次都使用 Retry 周期，使上游设备择时进行重试操作。在 PCI 总线中，有一个“16 Clock”原则，即 FRAME# 信号有效后，必须在 16 个时钟周期内置为无效，如果 PCI 桥发现来自上游设备的读总线事务不能在 16 个时钟周期内结束时，则使用 Retry 周期终止该总线事务。

3) PCI 桥 1 使用 Delayed 总线请求继续访问 PCI 设备 22。

4) PCI 桥 2 接收这个总线请求，并将这个 Delayed 总线请求继续传递。此时 PCI 桥 2 也将首先使用 Retry 周期，使 PCI 桥 1 择时重新发起相同的总线周期。此时 PCI 桥 2 的上游 PCI 总线被释放。

5) 这个数据请求最终到达 PCI 设备 22，如果 PCI 设备 22 没有将数据准备好时，也可以使用 Retry 周期，使 PCI 桥 2 择时重新发起相同的总线周期；如果数据已经准备好，PCI 设备 22 将接收这个数据请求，并将这个 Delayed 总线请求转换为 Delayed 总线完成事务。如果 Delayed 总线请求是读请求，则 Delayed 总线完成事务中含有数据，否则只有完成信息，而不包含数据。

6) Delayed 总线完成事务将“数据或者完成信息”传递给 PCI 桥 2，当 PCI 桥 1 重新发出 Non-Posted 总线请求时，PCI 桥 2 将这个“数据或者完成信息”传递给 PCI 桥 1。

7) HOST 主桥重新发出存储器读总线事务时，PCI 桥 1 将“数据或者完成信息”传递给 HOST 主桥，最终完成整个 PCI 总线事务。

由以上分析可知，Delayed 总线周期由 Delayed 总线请求和 Delayed 总线完成两部分组成。下面将 Delayed 读请求总线事务简称为 DRR (Delayed Read Request)，Delayed 读完成总线事务简称为 DRC (Delayed Read Completion)；而将 Delayed 写请求总线事务简称为 DWR

(Delayed Write Request), Delayed 写完成总线事务简称为 DWC (Delayed Write Completion)。

PCI 总线使用 Delayed 总线事务,在一定程度上可以提高 PCI 总线的利用率。因为在进行 Non-Posted 总线事务时,Non-Posted 请求在通过 PCI 桥之后,可以暂时释放 PCI 总线,但是采用这种方式,HOST/PCI 桥将会择时进行重试操作。在许多情况下,使用 Delayed 总线事务,并不能取得理想的效果,因为过多的重试周期也将大量消耗 PCI 总线的带宽。

为了进一步提高 Non-Posted 总线事务的执行效率,PCI-X 总线将 PCI 总线使用的 Delayed 总线事务,升级为 Split 总线事务。采用 Split 总线事务可以有效解决 HOST/PCI 桥的这些重试操作。Split 总线事务的基本思想是发送端首先将 Non-Posted 总线请求发送给接收端,然后再由接收端主动地将数据传递给发送端。

除了 PCI-X 总线可以使用 Split 总线事务进行数据传送之外,有些处理器,如 x86 和 PowerPC 处理器的 FSB (Front Side Bus) 总线也支持这种 Split 总线事务,因此这些 HOST 主桥也可以发起这种 Split 总线事务。在 PCIe 总线中,Non-Posted 数据传送都使用 Split 总线事务完成,而不再使用 Delayed 总线事务。在第 1.5.1 节将简要介绍 Split 总线事务和 PCI-X 总线对 PCI 总线的一些功能上的增强。

1.4 PCI 总线的中断机制

PCI 总线使用 INTA#、INTB#、INTC#和 INTD#信号向处理器发出中断请求。这些中断请求信号为低电平有效,并与处理器的中断控制器连接。在 PCI 体系结构中,这些中断信号属于边带信号 (Sideband Signals),PCI 总线规范并没有明确规定在一个处理器系统中如何使用这些信号,因为这些信号对于 PCI 总线是可选信号。PCI 设备还可以使用 MSI 机制向处理器提交中断请求,而不使用这组中断信号。有关 MSI 机制的详细说明见第 10 章。

1.4.1 中断信号与中断控制器的连接关系

不同的处理器使用的中断控制器不同,如 x86 处理器使用 APIC (Advanced Programmable Interrupt Controller) 中断控制器,而 PowerPC 处理器使用 MPIC (Multiprocessor Interrupt Controller) 中断控制器。这些中断控制器都提供了一些外部中断请求引脚 IRQ_PINx#。外部设备,包括 PCI 设备可以使用这些引脚向处理器提交中断请求。

但是 PCI 总线规范没有规定 PCI 设备的 INTx 信号如何与中断控制器的 IRQ_PINx#信号相连,这为系统软件的设计带来了一定的困难,为此系统软件使用中断路由表存放 PCI 设备的 INTx 信号与中断控制器的连接关系。在 x86 处理器系统中,BIOS 可以提供这个中断路由表,而在 PowerPC 处理器中 Firmware 也可以提供这个中断路由表。

在一些简单的嵌入式处理器系统中,Firmware 并没有提供中断路由表,系统软件开发者需要事先了解 PCI 设备的 INTx 信号与中断控制器的连接关系。此时外部设备与中断控制器的连接关系由硬件设计人员指定。

假设在一个处理器系统中,共有 3 个 PCI 插槽 (分别为 PCI 插槽 A、B 和 C),这些 PCI 插槽与中断控制器的 IRQ_PINx 引脚 (分别为 IRQW#、IRQX#、IRQY#和 IRQZ#) 可以按照图 1-5 所示的拓扑结构进行连接。

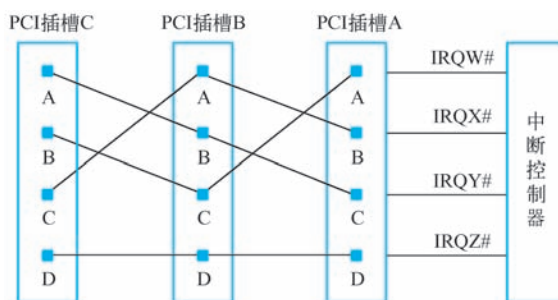


图 1-5 PCI 插槽与中断控制器连接拓扑图

采用图 1-5 所示的拓扑结构时，PCI 插槽 A、B、C 的 INTA#、INTB#和 INTC#信号将分散连接到中断控制器的 IRQW#、IRQX#和 IRQY#信号，而所有 INTD#信号将共享一个 IRQZ#信号。采用这种连接方式时，整个处理器系统使用的中断请求信号，其负载较为均衡。而且这种连接方式保证了每一个插槽的 INTA#信号都与一根独立的 IRQx#信号对应，从而提高了 PCI 插槽中断请求的效率。

在一个处理器系统中，多数 PCI 设备仅使用 INTA#信号，很少使用 INTB#和 INTC#信号，而 INTD#信号更是极少使用。在 PCI 总线中，PCI 设备配置空间的 Interrupt Pin 寄存器记录该设备究竟使用哪个 INTx 信号，该寄存器的详细介绍见第 2.3.2 节。

1.4.2 中断信号与 PCI 总线的连接关系

在 PCI 总线中，INTx 信号属于边带信号。所谓边带信号是指这些信号在 PCI 总线中是可选信号，而且只能在一个处理器系统的内部使用，并不能离开这个处理器环境。PCI 桥也不会处理这些边带信号。这给 PCI 设备将中断请求发向处理器带来了一些困难，特别是给挂在 PCI 桥之下的 PCI 设备进行中断请求带来了一些麻烦。

在一些嵌入式处理器系统中，这个问题较易解决。因为嵌入式处理器系统很清楚在当前系统中存在多少个 PCI 设备，这些 PCI 设备使用了哪些中断资源。在多数嵌入式处理器系统中，PCI 设备的数量小于中断控制器提供的外部中断请求引脚数，而且在嵌入式系统中，多数 PCI 设备仅使用 INTA#信号提交中断请求。

在这类处理器系统中，可能并不含有 PCI 桥，因而 PCI 设备的中断请求信号与中断控制器的连接关系较易确定。即便存在 PCI 桥，来自 PCI 桥之下的 PCI 设备的中断请求也较易处理。

在多数情况下，嵌入式处理器系统使用的 PCI 设备仅使用 INTA#信号进行中断请求，所以只要将这些 INTA#信号挂接到中断控制器的独立 IRQ_PIN#引脚上即可。这样每一个 PCI 设备都可以独占一个单独的中断引脚。

而在 x86 处理器系统中，这个问题需要 BIOS 参与来解决。在 x86 处理器系统中，有许多 PCI 插槽，处理器系统并不知道在这些插槽上将要挂接哪些 PCI 设备，也并不知道这些 PCI 设备是否需要使用所有的 INTx#信号线。因此 x86 处理器系统必须对各种情况进行处理。

x86 处理器系统还经常使用 PCI 桥进行 PCI 总线扩展，扩展出来的 PCI 总线还可能挂接一些 PCI 插槽，这些插槽上的 INTx#信号仍然需要处理。PCI 桥规范并没有要求桥片传递其下 PCI 设备的中断请求。事实上多数 PCI 桥也没有为下游 PCI 总线提供中断引脚 INTx#，管

理其下游总线的 PCI 设备。但是 PCI 桥规范推荐使用表 1-3 建立下游 PCI 设备的 INTx 信号与上游 PCI 总线 INTx 信号之间的映射关系。

表 1-3 PCI 设备 INTx#信号与 PCI 总线 INTx#信号的映射关系

设 备 号	PCI 设备的 INTx#信号	PCI 总线的 INTx#信号
0, 4, 8, 12, 16, 20, 24, 28	INTA#	INTA#
	INTB#	INTB#
	INTC#	INTC#
	INTD#	INTD#
1, 5, 9, 13, 17, 21, 25, 29	INTA#	INTB#
	INTB#	INTC#
	INTC#	INTD#
	INTD#	INTA#
2, 6, 10, 14, 18, 22, 26, 30	INTA#	INTC#
	INTB#	INTD#
	INTC#	INTA#
	INTD#	INTB#
3, 7, 11, 15, 19, 23, 27, 31	INTA#	INTD#
	INTB#	INTA#
	INTC#	INTB#
	INTD#	INTC#

下面举例说明该表的含义。在 PCI 桥下游总线上的 PCI 设备，如果其设备号为 0，那么这个设备的 INTA#引脚将和 PCI 总线的 INTA#引脚相连；如果其设备号为 1，其 INTA#引脚将和 PCI 总线的 INTB#引脚相连；如果其设备号为 2，其 INTA#引脚将和 PCI 总线的 INTC#引脚相连；如果其设备号为 3，其 INTA#引脚将和 PCI 总线的 INTD#引脚相连。

在 x86 处理器系统中，由 BIOS 或者 APCI 表记录 PCI 总线的 INTA~D#信号与中断控制器之间的映射关系，保存这个映射关系的数据结构也被称为中断路由表。大多数 BIOS 使用表 1-3 中的映射关系，这也是绝大多数 BIOS 支持的方式。如果在一个 x86 处理器系统中，PCI 桥下游总线的 PCI 设备使用的中断映射关系与此不同，那么系统软件程序员需要改动 BIOS 中的中断路由表。

BIOS 初始化代码根据中断路由表中的信息，可以将 PCI 设备使用的中断向量号写入到该 PCI 设备配置空间的 Interrupt Line register 寄存器中，该寄存器将在第 2.3.2 节中介绍。

1.4.3 中断请求的同步

在 PCI 总线中，INTx 信号是一个异步信号。所谓异步是指 INTx 信号的传递并不与 PCI 总线的数据传送同步，即 INTx 信号的传递与 PCI 设备使用的 CLK#信号无关。这个“异步”信号给系统软件的设计带来了一定的麻烦。

系统软件程序员需要注意“异步”这种事件，因为几乎所有“异步”事件都会带来系统的“同步”问题。以图 1-1 为例，当 PCI 设备 11 使用 DMA 写方式，将一组数据写入存

存储器，该设备在最后一个数据离开 PCI 设备 11 的发送 FIFO 时，会认为 DMA 写操作已经完成。此时这个设备将通过 INTx 信号，通知处理器 DMA 写操作完成。

此时处理器（驱动程序的中断服务例程）需要注意，因为 INTx 信号是一个异步信号，当处理器收到 INTx 信号时，并不意味着 PCI 设备 11 已经将数据写入存储器中，因为 PCI 设备 11 的数据传递需要通过 PCI 桥 x1 和 HOST 主桥，最终才能到达存储器控制器。

而 INTx 信号是“异步”发送给处理器的，PCI 总线并不知道这个“异步”事件何时被处理。很有可能处理器已经接收到 INTx 信号，开始执行中断处理程序时，该 PCI 设备还没有完全将数据写入存储器。

因为“PCI 设备向处理器提交中断请求”与“将数据写入存储器”分别使用了两个不同的路径，处理器系统无法保证哪个信息率先到达。从而在处理器系统中存在“中断同步”的问题，PCI 总线提供了以下两种方法解决这个同步问题。

(1) PCI 设备保证在数据到达目的地之后，再提交中断请求。

显然这种方法不仅加大了硬件的开销，而且也不容易实现。如果 PCI 设备采用 Posted 写总线事务，PCI 设备无法单纯通过硬件逻辑判断数据什么时候写入到存储器。此时为了保证数据到达目的地后，PCI 设备才能提交中断请求，PCI 设备需要使用“读刷新”的方法保证数据可以到达目的地，其方法如下。

PCI 设备在提交中断请求之前，向 DMA 写的区域发出一个读请求，这个读请求总线事务将被 PCI 设备转换为读完成总线事务，当 PCI 设备收到这个读完成总线事务后，再向处理器提交中断请求。PCI 总线的“序”机制保证这个存储器读请求，会将 DMA 数据最终写入存储器，有关 PCI 序的详细说明见第 11.3 节。

PCI 总线规范要求 HOST 主桥和 PCI 桥必须保证这种读操作可以刷新写操作。但问题是，没有多少芯片设计者愿意提供这种机制，因为这将极大地增加他们的设计难度。除此之外，使用这种方法也将增加中断请求的延时。

(2) 中断服务例程使用“读刷新”方法。

中断服务例程在使用“PCI 设备写入存储器”的这些数据之前，需要对这个 PCI 设备进行读操作。这个读操作也可以强制将数据最终写入存储器，实际上是将数据写到存储器控制器中。这种方法利用了 PCI 总线的传送序规则，与第 1 种方法基本相同，只是这种方法使用软件方式，而第 1 种方式使用硬件方式。第 11.3 节将详细介绍这个读操作如何将数据刷新到存储器中。

第 2 种方法也是绝大多数处理器系统采用的方法。程序员在编写中断服务例程时，往往都是先读取 PCI 设备的中断状态寄存器，判断中断产生原因之后，才对 PCI 设备写入的数据进行操作。这个读取中断状态寄存器的过程，一方面可以获得设备的中断状态，另一方面可以保证 DMA 写的最终数据到达存储器。如果驱动程序不这样做，就可能产生数据完整性问题。产生这种数据完整性问题的原因是 INTx 这个异步信号。

这里也再次提醒系统程序员注意 PCI 总线的“异步”中断所带来的数据完整性问题。在一个操作系统中，即便中断处理程序没有首先读取 PCI 设备的寄存器，也多半不会出现问题，因为在操作系统中，一个 PCI 设备从提交中断到处理器开始执行设备的中断服务例程，所需要的时间较长，处理器系统基本上可以保证此时数据已经写入存储器。

但是如果系统程序员不这样做，这个驱动程序依然有 Bug，尽管这些 Bug 因为各种机缘

巧合，始终不能够暴露出来，而一旦这些 Bug 被暴露出来将难以定位。为此系统程序员务必重视设计中的每一个实现细节，当然仅凭小心谨慎是远远不够的，因为重视细节的前提是充分理解这些细节。

PCI 总线 V2.2 规范还定义了一种新的中断机制，即 MSI 中断机制。MSI 中断机制采用存储器写总线事务向处理器系统提交中断请求，其实现机制是向 HOST 处理器指定的一个存储器地址写指定的数据。这个存储器地址一般是中断控制器规定的某段存储器地址范围，而且数据也是事先安排好的数据，通常含有中断向量号。

HOST 主桥会将 MSI 这个特殊的存储器写总线事务进一步翻译为中断请求，提交给处理器。目前 PCIe 和 PCI-X 设备必须支持 MSI 中断机制，但是 PCI 设备并不一定都支持 MSI 中断机制。

目前 MSI 中断机制虽然在 PCIe 总线上已经成为主流，但是在 PCI 设备中并不常用。即便是支持 MSI 中断机制的 PCI 设备，在设备驱动程序的实现中也很少使用这种机制。首先 PCI 设备具有 INTx# 信号可以传递中断，而且这种中断传送方式在 PCI 总线中根深蒂固。其次 PCI 总线是一个共享总线，传递 MSI 中断需要占用 PCI 总线的带宽，需要进行总线仲裁等一系列过程，远没有使用 INTx# 信号线直接。

但是使用 MSI 中断机制可以取消 PCI 总线这个 INTx# 边带信号，可以解决使用 INTx 中断机制所带来的数据完整性问题。而更为重要的是，PCI 设备使用 MSI 中断机制，向处理器系统提交中断请求时，还可以通知处理器系统产生该中断的原因，即通过不同中断向量号表示中断请求的来源。当处理器系统执行中断服务例程时，不需要读取 PCI 设备的中断状态寄存器，获得中断请求的来源，从而在一定程度上提高了中断处理的效率。本书将在第 10 章详细介绍 MSI 中断机制。

1.5 PCI-X 总线简介

PCI-X 总线仍采用并行总线技术。PCI-X 总线使用的大多数总线事务基于 PCI 总线，但是在实现细节上略有不同。PCI-X 总线将工作频率提高到 533 MHz，并首先引入了 PME（Power Management Event）机制。除此之外，PCI-X 总线还提出了许多新的特性。

1.5.1 Split 总线事务

Split 总线事务是 PCI-X 总线的一个重要特性。该总线事务替代了 PCI 总线的 Delayed 数据传送方式，从而提高了 Non-Posted 总线事务的传送效率。下面以存储器读为例，说明 PCI-X 设备如何使用 Split 总线事务。

PCI-X 总线在进行存储器读总线事务时，总线事务的发起方（Requester）使用 Split 总线事务与总线事务接收端（Completer）进行数据交换，其步骤如下。

- 1) Requester 向 Completer 发起存储器读请求总线事务。

- 2) 这个存储器读请求在到达 Completer 之前，可能会经过多级 PCI-X 桥。这些 PCI-X 桥使用 Split Response 周期结束当前总线事务，释放上游 PCI 总线。之后继续转发这个存储器读请求，直到 Completer 认领这个存储器读请求总线事务。

- 3) Completer 认领存储器读请求总线事务后，会记录 Requester 的 ID 号，并使用 Split

Response 周期结束存储器读请求总线事务。

4) Completer 准备好数据后，将重新申请总线，并使用存储器读完成总线事务主动地将数据传送给 Requester。在这个完成报文中包含 Requester 的 ID 号，因为完成报文使用 ID 路由而不是地址路由。

5) 这些完成报文根据 ID 路由方式，最终到达 Requester。Requester 从完成报文中接收数据并完成整个存储器读请求。

与 Delayed 总线事务相比，Requester 获得的数据是 Completer 将数据完全准备好后，由 Completer 主动传递的，而不是通过 Requester 通过多次重试获得的，因此能够提高 PCI-X 总线的使用效率。PCI-X 总线提出的 Split 总线事务被 PCIe 总线继承。

1.5.2 总线传送协议

PCI-X 总线改变了 PCI 总线使用的传送协议。目标设备可以将主设备发送的命令锁存，然后在下一个时钟周期进行译码操作。与 PCI 总线事务相比，PCI-X 总线采用的这种方式，虽然在总线时序中多使用了一个时钟周期，但是可以有效提高 PCI-X 总线的运行频率。

因为主设备通过数据线将命令发送到目标设备需要一定的延时。如果 PCI 总线频率较高，目标设备很难在一个时钟周期内接收完毕总线命令，并同时完成译码工作。而如果目标设备能够将主设备发出的命令先进行锁存，然后在下一个时钟周期进行译码则可以有效解决这个译码时间 Margin 不足的问题，从而提高 PCI-X 总线的频率。PCI-X 1.0 总线可以使用的最高总线频率为 133 MHz，而 PCI-X 2.0 总线可以使用的最高总线频率为 533 MHz，远比 PCI 总线使用的总线频率高。

除了信号传送协议外，PCI-X 总线在进行 DMA 读写时，可以不进行 Cache 共享一致性操作，而 PCI 总线进行 DMA 读写时必须进行 Cache 一致性操作。在某些特殊情况下，DMA 读写时进行 Cache 共享一致性不但不能提高总线传送效率，反而会降低。第 3.3 节将详细讨论与 Cache 一致性相关的 PCI 总线事务。

此外 PCI-X 总线还支持乱序总线事务，即 Relaxed Ordering，该总线事务被 PCIe 总线继承。对于某些应用，PCI-X 设备使用 Relaxed ordering 方式，可以有效地提高数据传送效率。但是支持 Relaxed Ordering 的设备，需要较多的数据缓存和硬件逻辑处理这些乱序，这为 PCI-X 设备的设计带来了不小的困难。

1.5.3 基于数据块的突发传送

在 PCI 总线中，一次突发传送的大小为 2 个以上的双字。一次突发传送所携带的数据越多，突发传送的总线利用率也越高。

而 PCI 总线的突发传送仍然存在缺陷。在 PCI 总线中，数据发送端知道究竟需要发送多少字节的数据，但是接收端并不清楚到底需要接收多少数据。这种不确定性，为接收端的缓冲管理带来了较大的挑战。

为此 PCI-X 总线使用基于数据块的突发传送方式，发送端以 ADB (Allowable Disconnect Boundary) 为单位，将数据发送给接收端，一次突发读写为一个以上的 ADB。采用这种方式，接收端可以事先预知是否有足够的接收缓冲，接收来自发送端的数据，从而可以及时断开当前总线周期，以节约 PCI-X 总线的带宽。在 PCI-X 总线中，ADB 的大小为 128B。

由于 ADB 的引入，PCI 总线与 Cache 相关的总线事务如 Memory Read Line、Memory Read Multiline 和 Memory Write and Invalidate，都被 PCI-X 总线使用与 ADB 相关的总线事务替代。因为通过 ADB，PCI-X 桥（HOST 主桥）可以准确地预知即将访问的数据在 Cache 中的分布情况。

PCI-X 总线还增加了一些其他特性，如在总线事务中增加传送字节计数，限制等待状态等机制，并增强了奇偶校验的管理方式。但是 PCI-X 总线还没有普及，就被 PCIe 总线替代。因此在 PC 领域和嵌入式领域很少有基于 PCI-X 总线的设备，PCI-X 设备仅在一些高端服务器上出现。因此本节不对 PCI-X 总线做进一步描述。事实上，PCI-X 总线的许多特性都被 PCIe 总线继承。

1.6 小结

本章主要介绍了 PCI 总线的基本组成部件，PCI 设备如何提交中断请求，以及 PCI-X 总线对 PCI 总线的功能增强。本章的重点在于 PCI 总线的 Posted 和 Non-Posted 总线事务，以及 PCI 总线如何使用 Delayed 传送方式处理 Non-Posted 总线事务，请读者务必深入理解这两种总线事务的不同。